MBK Knowledge Base

INTERNATIONAL SCHOOL MANAGEMENT SYSTEM

Documentation & Reference Guide

React · Vite · Supabase · Tailwind CSS

Covers: Tutorials · How-To Guides · Explanations · Design Documents

MBK International School â€” Knowledge Base

A complete school management system built with React, Vite, and Supabase. Manages students, teachers, parents, and supervisors across 9 feature domains.

Quick Start

Admin	Manage users, classes, exams, academic calendar, bulk uploads
Teacher	Enter exam results, record attendance, assign homework, create quizzes
Parent	View childrenâ€™s reports, take quizzes, monitor progress
Supervisor	Review lesson plans, verify exam entries, monitor teachers

Tutorials

Tutorial: Admin Getting Started

This tutorial walks you through the admin workflow â€” from logging in to entering your first exam result. Youâ€™ll learn the core navigation, understand the role-based dashboard, and complete a real task.

What youâ€™ll need

- Admin credentials (email + password)
- A browser (Chrome, Firefox, or Edge)
- The application URL (e.g., `http://localhost:5173` in development)

Step 1: Log in

1. Open the application URL.
2. Youâ€™ll see the login page with the MBK International School branding.
3. Enter your admin email and password.
4. Click **Sign In**.

Youâ€™ll land on the **Admin Dashboard** â€” the central hub showing key stats: total students, teachers, classes, and exam progress.

Step 2: Explore the dashboard

The dashboard displays:

- **Student count** â€” total enrolled students
- **Teacher count** â€” active teachers
- **Class count** â€” classes with assigned subjects
- **Exam progress** â€” percentage of exams entered this month

The sidebar on the left shows all admin navigation. Each section corresponds to a feature domain.

Step 3: Set up the academic year

Before entering exams, you need an active academic year and term.

1. Click **Academic** in the sidebar.
2. If no academic year exists, click **Create Year**.
3. Enter the year name (e.g., â€œ2025-2026â€), start date, and end date.
4. Toggle **Is Current** to on.
5. Create a term within that year (e.g., â€œTerm 1â€).
6. Select the months this term covers.

Now the system knows which term exams belong to.

Step 4: Assign teachers to classes

Teachers need class-subject assignments before they can enter grades.

1. Click **Class Subjects** in the sidebar.
2. Select a class (e.g., â€œGrade 5-Aâ€).
3. Select a subject (e.g., â€œMathematicsâ€).
4. Assign a teacher.
5. Click **Save**.

Repeat for each class-subject-teacher combination.

Step 5: Enter an exam result

Now try entering a studentâ€™s exam result:

1. Click **Exam Reports** or navigate to a class view.
2. Select the class, subject, month, and exam type (e.g., â€œCAâ€).
3. Find the student in the list.
4. Enter their score (e.g., 85) and total (e.g., 100).
5. Click **Save**.

The exam saves with status â€œpendingâ€ â€” supervisors verify these before they appear on reports.

Step 6: Verify it worked

1. Go back to the dashboard.
2. The exam progress counter should update.
3. The studentâ€™s parent can now see this score in their portal.

What you built

Youâ€™ve completed the core admin loop: set up academic structure â†’ assign teachers â†’ enter exams â†’ parents see results. Every other admin task builds on this foundation.

Tutorial: Teacher Getting Started

This tutorial walks you through the teacher workflow â€” from logging in to uploading your first exam results. Youâ€™ll learn how to manage your assigned classes, enter grades, and use the AI lesson planner.

What youâ€™ll need

- Teacher credentials (email + password)
- Assigned classes and subjects (set up by admin)
- A browser

Step 1: Log in

1. Open the application URL.
2. Enter your teacher email and password.

3. Click **Sign In**.

Youâ€™ll land on the **Teacher Dashboard** showing your assigned classes, subjects, and recent activity.

Step 2: View your classes

Your dashboard shows:

- **Assigned classes** â€” classes you teach
- **Assigned subjects** â€” subjects youâ€™re responsible for
- **Students** â€” student count per class

Click **Students** in the sidebar to see the full student list for your classes.

Step 3: Upload exam results

This is your primary daily task.

1. Click **Results** in the sidebar.
2. Select a class (e.g., â€œGrade 5-Aâ€).
3. Select a subject (e.g., â€œMathematicsâ€).
4. Select the month (e.g., â€œJanuaryâ€).
5. Select the exam type (e.g., â€œCAâ€ for continuous assessment).
6. For each student, enter:
    - **Score** â€” the studentâ€™s marks (e.g., 85)
    - **Total** â€” maximum marks (e.g., 100)
7. Click **Save**.

The results save with status â€œpendingâ€ â€” a supervisor must verify them before they appear on parent reports.

Step 4: Record attendance

1. Click **Attendance** in the sidebar.
2. Select the class and date.
3. For each student, mark:
    - **Present** â€” student attended
    - **Absent** â€” student was absent
    - **Late** â€” student arrived late
4. Add an optional note if needed.
5. Click **Save**.

Step 5: Assign homework

1. Click **Homework** in the sidebar.
2. Select the class and subject.
3. Enter a title (e.g., â€œChapter 3 Practice Problemsâ€).
4. Add a description (optional).
5. Set the due date.
6. Click **Assign**.

Students and parents see this in their portal.

Step 6: Create a quiz

1. Click **Quizzes** in the sidebar.
2. Click **Create Quiz**.
3. Select class, subject, and set a title.
4. Set open date, due date, and optional time limit.
5. Add questions â€” choose between:
   - **Multiple choice** â€” provide options and mark the correct answer
   - **Direct answer** â€” student types their response
6. Click **Publish**.

Students can now take the quiz from their portal.

Step 7: Use the AI lesson planner

1. Click **Lesson Plans** in the sidebar.
2. Click **Create Plan**.
3. Select the week, subject, and class.
4. For each period, fill in:
   - **Topic** â€” what youâ€™re teaching
   - **Objective** â€” learning objective
   - **Activities** â€” what students will do
5. Click **Submit for Review**.

The AI reviews your plan and scores it across 10 categories. A supervisor then does a final review.

What you built

Youâ€™ve completed the full teacher workflow: view classes â†’ enter grades â†’ record attendance â†’ assign homework â†’ create quizzes â†’ plan lessons. Every task flows from your assigned classes.

Tutorial: Parent Getting Started

This tutorial walks you through the parent portal â€” from logging in to viewing your childâ€™s exam report. Youâ€™ll learn how to monitor academic progress, view reports, and take quizzes.

What youâ€™ll need

- Parent credentials (email + password)
- At least one child enrolled in the school
- A browser

Step 1: Log in

1. Open the application URL.
2. Enter your parent email and password.
3. Click **Sign In**.

Youâ€™ll land on the **Parent Dashboard** showing your childrenâ€™s overview â€” names, classes, and recent activity.

Step 2: View your children

1. Click **Children** in the sidebar.
2. Youâ€™ll see a list of your enrolled children with:
    - Name and class
    - Recent exam scores
    - Attendance summary

Click any child to see their full profile.

Step 3: Check exam results

1. Click **Exam Results** in the sidebar.
2. Select a child from the dropdown.
3. Choose a month and term.
4. Youâ€™ll see all subjects with scores broken down by exam type:
    - CA (Continuous Assessment)
    - Homework
    - Classwork
    - Quiz
    - Attendance

Each subject shows the score, total, and percentage.

Step 4: View monthly reports

1. Click **Monthly Report** in the sidebar.
2. Select the child and month.
3. The report shows:
    - Subject-by-subject breakdown
    - Average score per subject
    - Class rank
    - Teacher comments

Step 5: View midterm reports

1. Click **Midterm Report** in the sidebar.
2. Select the child and term.
3. The midterm report includes:
    - Cumulative scores across the term
    - Subject rankings
    - Class average comparison
    - Overall position in class

Step 6: Take a quiz (if assigned)

Your childâ€™s teacher may assign quizzes. You can preview them:

1. Click **Quizzes** in the sidebar.
2. See available quizzes for your children.
3. Click a quiz to see the questions and answers.

Step 7: Check attendance

The attendance section shows daily attendance records:

- Green = present
- Red = absent
- Yellow = late

You can filter by date range and child.

What you built

Youâ€™ve learned the full parent workflow: view children â†’ check results â†’ read reports â†’ monitor attendance. You can now track your childâ€™s academic progress across all subjects and terms.

Tutorial: Supervisor Getting Started

This tutorial walks you through the supervisor workflow â€” from logging in to reviewing a lesson plan. Youâ€™ll learn how to verify exam entries, monitor teacher progress, and approve lesson plans.

What youâ€™ll need

- Supervisor credentials (email + password)
- A browser

Step 1: Log in

1. Open the application URL.
2. Enter your supervisor email and password.
3. Click **Sign In**.

Youâ€™ll land on the **Supervisor Dashboard** showing verification stats, teacher progress, and pending reviews.

Step 2: Monitor teacher exam entries

The dashboard shows which teachers have entered exams and which are missing.

1. Click **Verifications** in the sidebar.
2. Youâ€™ll see a table of teachers with:
    - Class and subject assigned
    - Month and exam type
    - Number of students entered vs.Â expected
    - Completion status

Teachers marked â€œincompleteâ€ havenâ€™t entered results for all students.

Step 3: Verify exam results

When a teacher submits exam results, they need your approval.

1. Click **Verifications** in the sidebar.
2. Review the pending exams.
3. For each entry:
    - Check the scores look reasonable
    - Verify the correct students are included
4. Click **Approve** to publish the results to parent reports.
5. Click **Reject** if something is wrong â€” the teacher must re-enter.

Approved results immediately appear in parent and student portals.

Step 4: Review lesson plans

Teachers submit lesson plans for your review.

1. Click **Lesson Plans** in the sidebar.
2. Youâ€™ll see plans in different statuses:
    - **Submitted** â€” awaiting your review
    - **In Review** â€” AI has scored it, pending your decision
    - **Approved** â€” published
    - **Rejected** â€” sent back to teacher
3. Click a plan to see:
    - The full weekly schedule with periods
    - AI review scores across 10 categories
    - AI executive summary and recommendations
    - Your comment field
4. Add your feedback and click **Approve** or **Request Revision**.

Step 5: View exam reports

1. Click **Reports** in the sidebar.
2. Select a class and term.
3. View aggregated exam data across all subjects.

Step 6: Take teacher actions

As a supervisor, you can also perform teacher-level actions:

- **Attendance** â€” record attendance for any class
- **Homework** â€” assign homework to any class
- **Quizzes** â€” create quizzes for any class
- **Grade Quizzes** â€” manually grade quiz attempts

These work exactly like the teacher workflow.

What you built

Youâ€™ve completed the supervisor loop: monitor teachers â†’ verify exams â†’ review lesson plans â†’ generate reports. Youâ€™re the quality

gatekeeper ensuring data accuracy and teaching standards.

How-To Guides

How to Set Up Academic Year and Terms

Configure the academic calendar so exams, reports, and attendance are tied to the correct time period.

Prerequisites

- Admin role
- Access to the Academic section

Steps

CREATE AN ACADEMIC YEAR

1. Click **Academic** in the sidebar.
2. Click **Create Year**.
3. Fill in:
   - **Name:** e.g., â€œ2025-2026â€
   - **Start Date:** first day of the academic year
   - **End Date:** last day of the academic year
   - **Is Current:** toggle ON (only one year can be current)
4. Click **Save**.

CREATE TERMS WITHIN THE YEAR

1. In the academic year view, click **Add Term**.
2. Fill in:
   - **Name:** e.g., â€œTerm 1â€, â€œTerm 2â€, â€œTerm 3â€
   - **Start Date:** first day of the term
   - **End Date:** last day of the term
   - **Is Current:** toggle ON for the active term
   - **Months:** select which calendar months this term covers (e.g., January, February, March)
3. Click **Save**.

CONFIGURE GRADE SCALES

1. In the Academic section, find **Grade Scales**.
2. Define score ranges and letter grades:

90	100	A	Excellent	4.0
80	89	B	Very Good	3.5
70	79	C	Good	3.0
60	69	D	Pass	2.5
0	59	F	Fail	0.0

3. Click **Save**.

CONFIGURE REPORT WEIGHTS

1. Find **Report Config** in the Academic section.
2. Set the weights for final grade calculation:
   - **CA Weight:** e.g., 30% (continuous assessment average)
   - **Midterm Weight:** e.g., 30%
   - **Final Weight:** e.g., 40%
3. The weights must sum to 100.
4. Click **Save**.

Verification

- The dashboard shows the current academic year and term.
- Teachers can only enter exams for the current termâ€™s months.
- Reports calculate using the configured weights.

Troubleshooting

Problem: Teachers canâ€™t enter exams for a month. **Fix:** Check that the month is included in the current termâ€™s month list.

Problem: Reports show wrong grades. **Fix:** Verify the grade scale ranges are correct and non-overlapping.

Problem: Two academic years are marked current. **Fix:** Only one year can be current â€” uncheck the old one first.

How to Create Class-Subject Assignments

Assign teachers to specific class-subject combinations so they can enter exam results and manage their classes.

Prerequisites

- Admin role
- Existing teachers in the system
- Existing classes and subjects

Steps

1. Click **Class Subjects** in the sidebar.

2. Youâ€™ll see a table of existing assignments.
3. Click **Add Assignment**.
4. Select:
   - **Class:** e.g., â€œGrade 5-Aâ€
   - **Subject:** e.g., â€œMathematicsâ€
   - **Teacher:** select from the dropdown of teachers with `assignedClasses` and `assignedSubjects` set
5. Click **Save**.

BULK ASSIGNMENTS

To assign one teacher to multiple classes:

1. Go to **Users** in the sidebar.
2. Find the teacher and click **Edit**.
3. Under **Assigned Classes**, check all classes they teach.
4. Under **Assigned Subjects**, check all subjects they teach.
5. Click **Save**.

Then create class-subject assignments â€” the teacher will appear in the dropdown for their assigned classes.

REMOVE AN ASSIGNMENT

1. In **Class Subjects**, find the assignment.
2. Click the delete icon.
3. Confirm removal.

This doesnâ€™t delete the teacher or class â€” it just removes the teaching assignment.

Verification

- The teacherâ€™s dashboard shows only their assigned classes.
- Teachers can only enter exam results for their assigned class-subject pairs.
- Supervisors see the assignment in the monitoring view.

Troubleshooting

Problem: Teacher doesnâ€™t appear in the class-subject dropdown. **Fix:** Check that the teacherâ€™s `assignedClasses` and `assignedSubjects` include the target class and subject.

Problem: Teacher canâ€™t see a class on their dashboard. **Fix:** Create a class-subject assignment linking that teacher to the class.

How to Enter and Verify Exam Results

The exam workflow has two stages: teachers enter results, then supervisors verify them. This guide covers both.

Prerequisites

- Teacher or admin role (for entering)
- Supervisor role (for verifying)

- Active academic year and term
- Class-subject assignments configured

Entering exam results (teacher)

1. Click **Results** in the sidebar.
2. Select from the filters:
   - **Class:** your assigned class
   - **Subject:** your assigned subject
   - **Month:** a month in the current term
   - **Exam Type:** CA, Homework, Classwork, Quiz, or Attendance
3. The student list loads for that class.
4. For each student:
   - Enter **Score** (e.g., 85)
   - Enter **Total** (e.g., 100)
   - Optionally add a **Comment**
5. Click **Save All**.

Results save with status `pending`.

EXAM TYPES

CA	Continuous Assessment	Yes (one of CA or Homework+Classwork)
Homework	Take-home work	Yes (with Classwork)
Classwork	In-class exercises	Yes (with Homework)
Quiz	Quick tests	Yes
Attendance	Presence tracking	Visible but not required
Midterm	Mid-term exam	Optional
Final	End-of-term exam	Optional

MONTHLY ENTRY RULES

- **Quiz** is always required for every student
- **Coursework** is required â€” satisfied by either:
  - CA for every student, OR
  - Both Homework and Classwork for every student
- **Attendance** is visible but not required

Verifying exam results (supervisor)

1. Click **Verifications** in the sidebar.
2. Review pending entries â€” grouped by teacher, class, subject, and month.
3. For each entry:
   - Check the number of students entered matches enrollment
   - Review score distributions for outliers

    - Verify correct exam type and month
4. Click **Approve** to publish to parent reports.
5. Click **Reject** with a reason if something is wrong.

Approved results appear in parent and student portals immediately.

Verification

- Teachers see â€œpendingâ€ status on entered results until verified.
- Supervisors see completion percentages in the monitoring view.
- Parents see results only after supervisor approval.

Troubleshooting

Problem: Teacher canâ€™t enter results for a month. **Fix:** Check the month is included in the current term. Only current term months are available.

Problem: Results donâ€™t appear on parent reports. **Fix:** A supervisor must approve the results first. Check the verification queue.

Problem: Wrong number of students shown. **Fix:** Verify the class-subject assignment and student enrollment match.

How to Create and Manage Quizzes

Build quizzes with multiple choice and direct answer questions, publish them to students, and grade attempts.

Prerequisites

- Teacher or admin role
- Assigned classes and subjects

Creating a quiz

1. Click **Quizzes** in the sidebar.
2. Click **Create Quiz**.
3. Fill in:
    - **Title:** e.g., â€œChapter 5 Review Quizâ€
    - **Class:** select the target class
    - **Subject:** select the subject
    - **Description:** optional context for students
    - **Open Date:** when students can start
    - **Due Date:** deadline for submissions
    - **Time Limit:** optional minutes (e.g., 30)
    - **Question Order:** â€œcreatedâ€ (fixed) or â€œrandomizedâ€
4. Click **Save as Draft** or **Publish**.

Adding questions

1. In the quiz editor, click **Add Question**.
2. Choose the question type:

MULTIPLE CHOICE

- Enter the **Prompt** (the question text)
- Add options (minimum 2):
  - **Label:** A, B, C, D
  - **Text:** the answer text
- Mark the **Correct Answer**
- Set **Points** (default: 1)

DIRECT ANSWER

- Enter the **Prompt**
- Set **Correct Answer** (for auto-grading)
- Optionally add a **Rubric** (for manual grading)
- Set **Points**

3. Repeat for each question.
4. Click **Save**.

Publishing

- **Draft:** only you can see it
- **Active:** students can take it
- **Closed:** submissions closed, results available

Set status to **Active** when ready for students.

Grading attempts

1. Click **Grade Quizzes** in the sidebar.
2. Select the quiz.
3. For each attempt:
   - Review the studentâ€™s answers
   - For multiple choice: auto-graded, review flagged items
   - For direct answer: manually grade using the rubric
   - Enter **Points Earned** for each question
4. Click **Save Grade**.

Attempt status changes from `submitted` to `graded`.

Viewing results

- **Quiz list** shows attempt counts and average scores
- **Grade Quizzes** shows individual student answers
- **Parent portal** shows quiz results for their children

Verification

- Students see the quiz in their portal during the open date window
- Scores appear on reports after grading
- Time-limited quizzes auto-submit when time expires

Troubleshooting

Problem: Students canâ€™t see the quiz. **Fix:** Check the quiz status is â€œActiveâ€ and today is between the open and due dates.

Problem: Time limit didnâ€™t work. **Fix:** Time limits require the student to have a browser open during the quiz. If they close and reopen, the timer continues from where it left off.

Problem: Canâ€™t grade a direct answer question. **Fix:** The rubric field guides your grading â€” thereâ€™s no auto-grading for direct answer questions. Enter points manually.

How to Record Student Attendance

Track daily attendance for each class. Attendance records are visible on student reports but not required for exam entry.

Prerequisites

- Teacher or admin role
- Assigned classes

Steps

1. Click **Attendance** in the sidebar.
2. Select the **Class** (e.g., â€œGrade 5-Aâ€).
3. Select the **Date** (defaults to today).
4. For each student, click one of:
    - **Present** (green) â€” student attended
    - **Absent** (red) â€” student was absent
    - **Late** (yellow) â€” student arrived late
5. Optionally add a **Note** (e.g., â€œLeft early for appointmentâ€).
6. Click **Save**.

Viewing attendance history

1. In the Attendance section, use the date picker to navigate to past dates.
2. The view shows the attendance record for that day.
3. You can update records if needed (before the date is finalized).

Attendance on reports

- Monthly reports include attendance counts (present/absent/late)
- Parents see attendance in their portal
- Attendance is tracked per class per day

Verification

- Each student gets one record per day per class
- The attendance section shows completion status
- Parents can view attendance in their portal

Troubleshooting

Problem: Student not showing in attendance list. **Fix:** Verify the student is enrolled in the selected class. Check the studentâ€™s `className` field matches.

Problem: Canâ€™t edit past attendance. **Fix:** Past attendance records may be locked. Contact an admin to unlock if needed.

How to Generate Student Reports

The system generates three types of reports: monthly, midterm, and final. Each aggregates exam data differently.

Prerequisites

- Exam results entered and approved
- Academic year and terms configured
- Grade scales defined

Monthly reports

Shows one month of exam data per subject.

1. Click **Monthly Report** (parent) or **Exam Reports** (admin/teacher).
2. Select the **Student** and **Month**.
3. The report shows per subject:
    - Each exam type score (CA, Homework, Classwork, Quiz, Attendance)
    - Average score for the month
    - Class average for comparison

HOW MONTHLY AVERAGE IS CALCULATED

```
Monthly Average = Sum of all exam scores / Number of exams
```

Each exam type contributes equally to the monthly average.

Midterm reports

Aggregates across multiple months in a term.

1. Click **Midterm Report**.
2. Select the **Student** and **Term**.
3. The report shows:
    - Per-subject scores across the term

- Subject rank (position among classmates)
- Class average per subject
- Highest score in class
- Overall rank among all students

HOW MIDTERM SCORE IS CALCULATED

```
Midterm Score = Average of monthly averages for the term
```

Final reports

Combines CA, midterm, and final exam with configurable weights.

1. Click **Final Report**.
2. Select the **Student** and **Term**.
3. The report shows:
   - CA average (weighted)
   - Midterm score (weighted)
   - Final exam score (weighted)
   - Total final score
   - Letter grade (A-F)
   - Pass/Fail status
   - Teacher comment
   - Principal comment

HOW FINAL SCORE IS CALCULATED

```
Final Score = (CA Average x CA Weight) + (Midterm x Midterm Weight) + (Final Exam x Final Weight)
```

Default weights: CA 30%, Midterm 30%, Final 40%.

GRADE SCALE

90-100	A	Excellent
80-89	B	Very Good
70-79	C	Good
60-69	D	Pass
0-59	F	Fail

Passing threshold: 60.

Verification

- Monthly reports update immediately when new exams are approved
- Midterm reports recalculate when any month in the term changes
- Final reports reflect the latest grade scale and weight configuration

Troubleshooting

Problem: Report shows no data. **Fix:** Check that exams exist for the selected student, month/term, and that theyâ€™re approved (status = â€œapprovedâ€).

Problem: Wrong grade on final report. **Fix:** Verify the report config weights sum to 100 and the grade scale is correct.

Problem: Student rank seems wrong. **Fix:** Rank is calculated across all students in the same class. Verify the studentâ€™s `className` matches the class being compared.

How to Use the AI Lesson Planner

Create lesson plans, get AI-powered feedback, and submit for supervisor review.

Prerequisites

- Teacher role
- Assigned classes and subjects

Creating a lesson plan

1. Click **Lesson Plans** in the sidebar.
2. Click **Create Plan**.
3. Fill in:
   - **Title:** e.g., â€œWeek 12 - Fractions and Decimalsâ€
   - **Subject:** select from your assigned subjects
   - **Class:** select from your assigned classes
   - **Week Label:** e.g., â€œWeek 12â€
4. Click **Save**.

Adding periods

The lesson plan covers a school week (Saturday through Wednesday, 5 days).

1. In the plan editor, click a day tab (Saturday, Sunday, etc.).
2. For each period, fill in:
   - **Period Number:** 1-6 (or however many periods your school has)
   - **Topic:** what youâ€™re teaching (e.g., â€œIntroduction to Fractionsâ€)
   - **Objective:** learning objective (e.g., â€œStudents will identify numerator and denominatorâ€)
   - **Activities:** what students will do (e.g., â€œWorksheet exercises, group discussionâ€)
   - **Slide Number:** optional reference to presentation slides
   - **Is Free:** toggle if the period is free (no teaching)
3. Click **Save Periods**.

PERIOD DETAILS

For each period, you can add structured activities:

1. Click **Add Activity** on a period.

2. Fill in:
    - **Activity:** description
    - **Time:** duration (e.g., â€œ15 minâ€)
    - **Resource:** materials needed
    - **Place:** location (e.g., â€œClassroomâ€, â€œLabâ€)

Submitting for AI review

1. After filling in all periods, click **Submit for Review**.
2. The system sends your plan to the AI review edge function.
3. Wait for the review (typically 10-30 seconds).

AI REVIEW SCORES

The AI evaluates your plan across 10 categories:

Learning Objectives	Clarity, specificity, measurability
Lesson Structure	Flow, timing, transitions
Student Engagement	Active learning, participation opportunities
Teaching Strategies	Variety, appropriateness
Differentiation	Adapting for different learners
Assessment Methods	How learning is checked
Curriculum Alignment	Matches expected standards
Classroom Management	Behavior, time management
Resources Materials	Appropriateness, availability
Overall Quality	Holistic assessment

Each category gets a score (0-100) and an explanation.

PERFORMANCE LEVELS

90-100	Excellent
80-89	Very Good
70-79	Good
60-69	Needs Improvement
0-59	Requires Significant Revision

Supervisor review

After AI review, the plan goes to your supervisor:

1. Supervisor sees the AI scores and executive summary.

2. They add their own comments.
3. They approve or request revision.
4. If revision is requested, you edit and resubmit.

Verification

- Plan status changes: `draft` -> `submitted` -> `in_review` -> `approved` / `rejected`
- AI review scores appear in the plan detail view
- Supervisors see all pending plans in their queue

Troubleshooting

Problem: AI review failed. **Fix:** Check that all periods have a topic and activities filled in. The AI needs content to evaluate.

Problem: Canâ€™t submit for review. **Fix:** Ensure the plan has at least one period with content. Empty plans canâ€™t be reviewed.

Problem: Scores seem low. **Fix:** Read the AIâ€™s improvement suggestions. Focus on clearer objectives, more varied activities, and better assessment methods.

Explanations

Architecture Overview

Why the system is built this way, and how the pieces fit together.

The problem

Schools need a centralized system to manage students, teachers, exams, and reports. Existing solutions are either too complex (enterprise SaaS) or too simple (spreadsheets). MBK International School needs something in between: a web app that handles the full academic workflow while staying simple enough for non-technical staff.

The approach

FRONTEND: REACT + VITE + TAILWIND

Why React: Component-based UI fits the dashboard pattern â€” each role has different views, but they share common UI elements (tables, forms, modals). Reactâ€™s composition model makes this natural.

Why Vite: Fast dev server, single-file output via `vite-plugin-singlefile`. The single HTML file deployment means the app can be hosted anywhere â€” even a static file server â€” without a build pipeline.

Why Tailwind CSS: Utility-first CSS eliminates the need for a custom design system. The theming system (Acanthus, Baroque, Aurora) uses CSS custom properties that Tailwindâ€™s `@theme` directive can reference.

BACKEND: SUPABASE

Why Supabase: It provides PostgreSQL, authentication, row-level security, and edge functions in one platform. For a school app, this means:

- **PostgreSQL:** relational data model fits students -> exams -> reports perfectly
- **Auth:** email/password login with session management
- **RLS:** teachers can only see their own classes, parents only their children
- **Edge Functions:** the AI lesson plan review runs as a Supabase edge function

CLIENT-SIDE ROUTING

Why not a router library: The app uses a simple `switch` statement on `currentPath` in `App.tsx`. Each role has its own route set. This is intentional â€” the app is small enough that a full router library would add complexity without benefit.

The routing works like this:

```
User logs in -> RoleContext determines role -> App.tsx switches on path -> correct page renders
```

STATE MANAGEMENT

Why React Query: Server state (exams, students, reports) is cached and refetched automatically. Local state (UI toggles, form inputs) stays in component state. No global store needed.

```
Server state -> TanStack React Query (cache, refetch, background sync)
UI state -> useState/useContext (RoleContext, ToastContext)
```

Key design decisions

ROLE-BASED ACCESS AT THE COMPONENT LEVEL

Each role has its own set of pages. The `AppContent` component checks `session.role` and renders the appropriate route set. This means:

- Admin sees all management pages
- Teacher sees class-specific pages
- Parent sees child-specific pages
- Supervisor sees monitoring pages

Thereâ€™s no shared routing table â€” each roleâ€™s routes are explicit.

EXAMS AS THE CENTRAL ENTITY

Everything revolves around exams:

```
Students -> take Exams -> per Subject -> per Month -> in Classes
Teachers -> enter Exams -> for their assigned Classes
Supervisors -> verify Exams -> before Parents see them
Parents -> view Exam results -> on Reports
```

This flow determines the entire data model and UI structure.

SINGLE-FILE DEPLOYMENT

The `vite-plugin-singlefile` plugin inlines all JavaScript, CSS, and assets into a single HTML file. This means:

- No CDN or asset server needed
- Can be emailed as an attachment
- Works offline after first load
- Simple deployment: just serve the HTML file

Trade-offs

What was gained: - Simple deployment (single HTML file) - Fast development (Vite HMR) - Type safety (TypeScript throughout) - Security (Supabase RLS)

What was given up: - Server-side rendering (SEO doesnâ€™t matter for a school app) - Code splitting (single file means everything loads at once) - Traditional routing (no URL-based navigation, no browser back button) - Offline-first (requires network for Supabase calls)

Alternatives considered

Next.js: Would add SSR and routing, but the single-file deployment goal conflicts with Next.jsâ€™s server-side model. The app doesnâ€™t need SEO.

Firebase: Similar to Supabase but less SQL-friendly. The relational data model (students -> exams -> reports) benefits from PostgreSQLâ€™s JOIN capabilities.

Custom backend: More control but more maintenance. Supabase handles auth, database, and edge functions â€” the three things a school app needs.

Data Model Design

Why the database is structured the way it is, and how the entities relate.

The problem

A school management system needs to track: - Users (admin, teacher, parent, supervisor) - Students and their class enrollments - Exam results across subjects, months, and exam types - Academic calendar (years, terms, months) - Assignments (which teacher teaches which class-subject) - Reports (aggregated exam data)

The challenge: these entities have complex relationships, and the access patterns differ by role.

The approach

CORE ENTITY RELATIONSHIP

```
profiles (users)
  +-- students (one profile -> many students via parentId)
  +-- exams (one teacher -> many exams via teacherId)
  +-- class_subjects (one teacher -> many assignments)
  +-- auth_id -> auth.users (Supabase auth)

students
  +-- exams (one student -> many exams)
  +-- attendance (one student -> many records)
  +-- parentId -> profiles (link to parent)

class_subjects
  +-- className + subjectId -> subjects
  +-- teacherId -> profiles
```

WHY PROFILES INSTEAD OF USERS

The `profiles` table stores application-level user data. Supabaseâ€™s `auth.users` handles authentication. Theyâ€™re linked via `auth_id`:

```
auth.users (Supabase)     profiles (Application)
+-- id (UUID)             +-- id (TEXT)
+-- email                 +-- auth_id -> auth.users.id
+-- password_hash         +-- name
                          +-- role
                          +-- assignedClasses
                          +-- assignedSubjects
```

This separation means: - Auth is handled by Supabase (secure, maintained) - Application data is in our control (flexible schema) - The `auth_id` link allows session restoration

WHY EXAMS ARE THE CENTRAL ENTITY

The exam table connects everything:

```
exam
+-- studentId -> students
+-- teacherId -> profiles
+-- subject (text, denormalized)
+-- examType (CA, Homework, Classwork, Quiz, Midterm, Final)
+-- month (text)
+-- status (pending -> approved/rejected)
+-- termId -> terms (optional)
```

The `status` field creates a workflow: teachers enter -> supervisors approve -> parents see. This two-step process ensures data quality.

WHY CLASS_SUBJECTS EXISTS

Teachers donâ€™t just teach â€œMathematicsâ€ â€” they teach â€œMathematics to Grade 5-Aâ€. The `class_subjects` table captures this:

```
class_subjects
+-- className: "Grade 5-A"
+-- subjectId -> subjects
+-- teacherId -> profiles
```

This enables: - Teachers see only their assigned classes - Supervisors monitor specific class-subject combinations - Reports aggregate by class-subject

RLS POLICIES

Row-Level Security ensures each role sees only what they should:

```
Teacher: WHERE teacherId = auth.uid()
  -> can only see their own exam entries

Parent:  WHERE parentId = auth.uid()
  -> can only see their children's exams

Student: WHERE studentId = auth.uid()
  -> can only see their own exams
```

Supervisors bypass RLS â€” they need to see all data for verification.

Trade-offs

Denormalized subject on exams: The `exams` table stores `subject` as text, not a foreign key. This duplicates data but simplifies queries â€” no JOIN needed for basic exam listing.

Text IDs: The app uses TEXT for primary keys (nanoid or similar). UUIDs would be more standard but TEXT is simpler and works with Supabase.

Term linkage is optional: Exams can exist without a `termId`. This allows flexibility but means reports must handle orphaned exams.

Alternatives considered

Normalized subject: Using `subjectId` as a foreign key on exams would prevent typos but add a JOIN to every exam query. The denormalized approach was chosen for query simplicity.

Separate exam_types table: An enum or lookup table for exam types would be more normalized but the types are fixed and small â€” an enum in the CHECK constraint is sufficient.

Soft deletes: Adding a `deletedAt` timestamp would allow recovery but adds complexity. The current approach uses hard deletes with CASCADE.

The Theming System

How the four-theme, light/dark variant system works, and why itâ€™s built on CSS custom properties.

The problem

Schools have different brand identities. A single â€œblue and whiteâ€ theme doesnâ€™t work for everyone. The system needs multiple themes that

can be switched without changing any component code.

The approach

FOUR THEMES, TWO VARIANTS EACH

Acanthus	Deep green (#0F4C3A)	Gold (#C8A24A)	(fleur)	Classical, academic
Baroque	Deep red (#5B1F1F)	Gold (#D4AF37)	(fleur)	Ornate, formal
Aurora	Purple (#7C3AED)	Cyan (#22D3EE)	(star)	Modern, vibrant
Light variants	Same palette	Adjusted contrast	Same	Print-friendly

Each theme has a dark mode (default) and a light mode.

CSS CUSTOM PROPERTIES

The entire theme is defined as CSS variables on `:root`:

```
:root {
  --bg: #0B0F14;
  --surface: #111827;
  --primary: #0F4C3A;
  --accent: #C8A24A;
  --text: #F5F5F4;
  --heading-font: "Cinzel", serif;
  --body-font: "Inter", sans-serif;
}
```

Components reference these variables, not hardcoded colors:

```
h1 { color: var(--accent); }
.card { background: var(--surface); border: 1px solid var(--border); }
```

THEME SWITCHING

The `ThemeSwitcher` component:

1. Reads the stored theme from `localStorage`
2. Sets `data-theme` and `data-mode` attributes on `<html>`
3. CSS selectors activate the right variable set:

```
html[data-theme="acanthus"] { --primary: #0F4C3A; --accent: #C8A24A; }
html[data-theme="baroque"] { --primary: #5B1F1F; --accent: #D4AF37; }
html[data-theme="aurora"] { --primary: #7C3AED; --accent: #22D3EE; }

html[data-mode="light"] { --bg: #F7F1E3; --text: #17211C; }
```

PAGE GRADIENT

Each theme has a subtle radial gradient background:

```
--page-gradient: radial-gradient(
  circle at 18% 0%,
  rgba(200, 162, 74, 0.11),
  transparent 32rem
), radial-gradient(
  circle at 85% 14%,
  rgba(15, 76, 58, 0.28),
  transparent 28rem
), linear-gradient(135deg, #080b10, #0B0F14, #111827);
```

This creates a unique ambient glow per theme without images.

SVG BODY PATTERN

A decorative SVG pattern is overlaid on the body:

```
body::before {
  background-color: var(--accent);
  -webkit-mask-image: url("data:image/svg+xml,...");
  mix-blend-mode: overlay;
  opacity: var(--pattern-opacity);
}
```

The pattern is a stylized leaf/fleur motif thatâ€™s themed via `--accent` color and `--pattern-opacity`.

ORNAMENT WATERMARK

Each theme has a signature character displayed as a subtle watermark:

```
#root::before {
  content: var(--ornament);
  color: var(--accent);
  opacity: 0.07;
}
```

Why this design

CSS variables over SASS/LESS: Variables are runtime-switchable. SASS compiles to static CSS â€” no theme switching without reloading.

Data attributes over classes: `data-theme="acanthus"` is more semantic than `class="theme-acanthus"` and avoids conflicts with Tailwindâ€™s class system.

Component-level theming: Each component reads from CSS variables. No theme context, no prop drilling, no HOC wrappers. The theme â€œjust worksâ€ because the CSS cascade handles it.

Trade-offs

What was gained: - Theme switching without component changes - Print-friendly light variants - Unique visual identity per theme - No JavaScript theme logic in components

What was given up: - Runtime theme customization (themes are defined in CSS, not configurable by users) - Per-component theme overrides (global theme only) - Dynamic theme creation (new themes require CSS changes)

Alternatives considered

CSS-in-JS (styled-components, emotion): Would allow runtime theming but adds bundle size and complexity. The CSS variable approach is simpler and faster.

Tailwindâ€™s dark mode: Tailwind has `dark:` variants but they only support two modes (light/dark). The four-theme system needs more granularity.

Theme provider context: A React context with theme values would work but adds re-renders on theme change. CSS variables change instantly without React involvement.

AI Lesson Plan Review

How the AI-powered lesson plan review system works, from submission to scoring.

The problem

Supervisors canâ€™t review every lesson plan in detail. Teachers submit weekly plans, and manual review is time-consuming. The system needs automated quality feedback thatâ€™s consistent and actionable.

The approach

ARCHITECTURE

```
Teacher submits plan
  -> Client sends periods to edge function
  -> Edge function calls LLM (GPT-4 or similar)
  -> LLM returns structured JSON scores
  -> Edge function saves to ai_reviews table
  -> Supervisor sees AI scores + adds their own review
```

THE EDGE FUNCTION

Located in `supabase/functions/review-lesson-plan/`, the edge function:

1. Receives the planâ€™s periods as JSON
2. Constructs a system prompt with scoring criteria
3. Calls the LLM API
4. Parses the structured JSON response
5. Saves the review to the database

SCORING CATEGORIES

The AI evaluates 10 categories, each scored 0-100:

Learning Objectives	High	Are objectives specific, measurable, achievable?
Lesson Structure	High	Does the lesson flow logically? Are transitions smooth?
Student Engagement	Medium	Are there active learning opportunities?
Teaching Strategies	Medium	Is there variety in instruction methods?
Differentiation	Medium	Are there adaptations for different learners?
Assessment Methods	High	How is student learning checked?
Curriculum Alignment	High	Does it match expected standards?
Classroom Management	Low	Are behavior and time management addressed?
Resources Materials	Low	Are materials appropriate and available?
Overall Quality	High	Holistic assessment of the plan

THE PROMPT STRUCTURE

The system prompt asks the LLM to:

1. Read each periodâ€™s topic, objective, and activities
2. Score each category on a 0-100 scale
3. Provide an explanation for each score
4. List strengths and improvement areas
5. Write an executive summary
6. Recommend a status (approve/revise/reject)

RESPONSE FORMAT

The LLM returns structured JSON:

```
{
  "category_scores": {
    "learning_objectives": { "score": 85, "explanation": "..." },
    "lesson_structure": { "score": 78, "explanation": "..." }
  },
  "total_score": 79,
  "percentage": 79,
  "performance_level": "good",
  "strengths": ["Clear objectives", "Varied activities"],
  "improvements": [
    { "area": "Assessment", "why": "...", "recommendation": "..." }
  ]
}
```

PERFORMANCE LEVELS

90-100	Excellent	Approve
80-89	Very Good	Approve with minor notes
70-79	Good	Approve or revise
60-69	Needs Improvement	Revise
0-59	Requires Significant Revision	Reject

SUPERVISOR WORKFLOW

1. AI review is saved automatically on submission
2. Supervisor opens the plan and sees AI scores
3. Supervisor reads the AIâ€™s executive summary
4. Supervisor adds their own comment
5. Supervisor approves or requests revision

The AI doesnâ€™t replace the supervisor â€” it provides consistent, structured feedback that speeds up the review.

Why this design

Edge function over client-side: The LLM API key stays server-side. The edge function handles retries, timeouts, and error formatting.

Structured JSON over free text: Structured output enables: - Consistent scoring across plans - Trend tracking over time - Comparison between teachers - Automated status recommendations

10 categories over a single score: Granular feedback helps teachers improve specific areas. A single â€œ79/100â€ doesnâ€™t tell them what to fix.

Trade-offs

What was gained: - Consistent, automated quality feedback - Scalable review process - Data-driven improvement tracking - Supervisor time savings

What was given up: - LLM cost per review (typically $0.01-0.05) - Dependency on external API availability - Potential for biased scoring (mitigated by structured criteria) - Real-time review (takes 10-30 seconds)

Alternatives considered

Rule-based scoring: Would be free and instant but too rigid. Lesson plans are creative â€” rules canâ€™t capture quality nuance.

Peer review: Teachers reviewing each otherâ€™s plans. Good for culture but doesnâ€™t scale and introduces social dynamics.

Supervisor-only review: The status quo. Works but doesnâ€™t scale â€” supervisors become bottlenecks.

Design Documents

Attendance & Homework Creation â€” Design Doc

Problem

The `attendance` and `homework` tables exist in the Supabase DB but have no creation UI. The StreamsPage (`/streams`) and parent dashboard query them and always show â€œNo records available.â€ Meanwhile, `UploadResults.tsx` stores Attendance as an *exam type* in the `exams` table â€” thatâ€™s a separate track for teacher-exam-entry progress reporting, not these per-student stream records.

Principles

1. **Roll-call attendance** â€” students default to present; teacher only marks absent/late.
2. **Homework is class-level with per-student overrides** â€” one assignment targets a whole class; teacher can tweak due dates or add notes per student.
3. **New sidebar pages** â€” separate from Streams view, distinct from Upload Results.
4. **Roles** â€” Teachers, Admin, and Supervisors all get both pages.

Feature 1: Record Attendance

DATA MODEL (EXISTI `ATTENDANCE` TABLE)

```
AttendanceRecord {
  id: string          // auto-generated
  studentId: string   // FK -> students.id
  className: string
  date: string        // YYYY-MM-DD
  status: 'present' | 'absent' | 'late'
  note?: string
  teacherId: string   // FK -> profiles.id
  createdAt: string
}
```

UI FLOW (PER-CLASS NAVIGATION)

1. Teacher opens **Record Attendance** from sidebar
2. Sees a class selector dropdown (only classes theyâ€™re assigned to; admin sees all)
3. Picks a class -> date picker (defaults to today) -> **Load** button
4. Page loads a table of all students in that class, each row showing:
   - Student name
   - Status dropdown (Present / Absent / Late) â€” default **Present** for all
   - Optional note field
5. Controls:
   - **Mark all absent** (bulk override row by row)
   - **Mark all late** (bulk override row by row)
   - **Reset all to present** (one click)
   - **Save** â€” inserts `attendance` rows only for non-present students (saves writes). If ALL students are Present, inserts a single summary

marker row or skips (teacher sees â€œAll present â€” nothing to saveâ€)

6. After save: toast success, optionally keep form for next entry

EDGE CASES

- **Duplicate date+student**: warn â€œAttendance already recorded for [student] on [date]. Overwrite?â€ then upsert.
- **No students in class**: show empty state â€œNo students enrolled in this class.â€
- **Weekend/holiday**: free-text note field lets teacher explain.

Feature 2: Assign Homework

DATA MODEL (EXISTI `HOMEWORK` TABLE)

```
HomeworkRecord {
  id: string
  studentId: string   // FK -> students.id
  className: string
  subject: string
  title: string
  description?: string
  dueDate: string
  status: 'assigned' | 'submitted' | 'graded'
  teacherId: string
  createdAt: string
}
```

UI FLOW (TWO-TAB LAYOUT, MATCHING UPLOADRESULTS STEP-INDICATOR)

Tab 1: View Assignments - Select class -> see all homework assigned to that class - Each row shows title, subject, due date, student count - Edit: click to modify title/description/due date - Delete: confirm dialog, removes all student rows for that assignment

Tab 2: New Assignment (two-step, step-indicator like UploadResults) - Step 1 (Config): - Class dropdown (assigned for teacher/supervisor, all for admin) - Subject dropdown (pulls from assigned subjects via `class_subjects` join, stores name string) - Title (required), Description (optional), Due date (required) - Step 2 (Students): - Table of all students in the class - Per-row overrides: Due date (prefilled editable), Notes (optional), Include? checkbox - Bulk: â€œSet all due dates to [date]â€ button - **Assign** button -> inserts one `homework` row per included student - After save: toast success, offer â€œAssign anotherâ€

EDGE CASES

- **Duplicate title+class+subject**: warn before creating duplicates
- **No students in class**: show empty state
- **Exempt all students**: block assign, â€œNo students selectedâ€

Routing & Sidebar

NEW ROUTES

Record Attendance	`/teacher/attendance`	teacher, admin, supervisor
Assign Homework	`/teacher/homework`	teacher, admin, supervisor

Admin sees them under different paths since admin routing uses `/admin/*`: - Admin: `/admin/attendance`, `/admin/homework` - Supervisor: `/supervisor/attendance`, `/supervisor/homework`

SIDEBAR NAV ITEMS

Add to admin, teacher, and supervisor nav arrays in `Sidebar.tsx`: - `{ label: 'Record Attendance', icon: CalendarCheck, path: '/<role>/attendance' }` - `{ label: 'Assign Homework', icon: BookOpenCheck, path: '/<role>/homework' }`

APP.TSX ROUTES

Add cases to the teacher, admin, and supervisor route switches.

DB layer

New file `src/lib/db/attendance.ts` and `src/lib/db/homework.ts` (or extend existing `streams.ts`):

```
// attendance.ts
export async function getStudentsAttendanceStatus(classNames: string[], date: string): Promise<AttendanceRecord[]>
export async function upsertAttendanceBatch(records: { studentId: string; status: string; note?: string }[]): Promise
export async function hasAttendanceForDate(className: string, date: string): Promise<boolean>

// homework.ts
export async function createHomeworkBatch(records: HomeworkRecord[]): Promise<void>
export async function getHomeworkByClass(className: string): Promise<HomeworkRecord[]>
export async function deleteHomework(id: string): Promise<void>
export async function updateHomework(id: string, data: Partial<HomeworkRecord>): Promise<void>
```

Implementation order

1. `src/lib/db/attendance.ts` â€” DB functions for upserting attendance batch
2. `src/pages/teacher/RecordAttendance.tsx` â€” the attendance UI page
3. Wire into `Sidebar.tsx` and `App.tsx` routes for teacher, admin, supervisor
4. `src/lib/db/homework.ts` â€” DB functions for homework batch create
5. `src/pages/teacher/AssignHomework.tsx` â€” the homework UI page
6. Wire sidebar + routes for homework
7. Update `supabase-schema.sql` with table definitions for `attendance` and `homework` (currently missing from the file â€” they exist live but not in version control)